



SDD Workflow Từ Spec Đến Code

5 pha · 1 triết lý · Không còn đoán mò

Chương 6 · Spec-Driven Development

GIỚI THIỆU CHƯƠNG

Động Cơ Của SDD Workflow

Chương 5 đã cho bạn ngôn ngữ — EARS Notation, 8 thành phần, Risk Matrix. Chương này cho bạn động cơ — một quy trình làm việc hoàn chỉnh biến spec thành production code thông qua năm pha được thiết kế kỹ lưỡng.

Tỷ lệ 40/60 lý thuyết–thực hành có chủ đích: bạn sẽ thực sự chạy workflow, không chỉ đọc về nó. Ba công cụ cụ thể được giới thiệu theo thứ tự tăng dần về tính linh hoạt — không có "cách tốt nhất", chỉ có "cách phù hợp nhất" với đội nhóm và dự án của bạn.



"Sai ở đâu, sửa ở Spec đó."

Fix the Spec, not the Code.

Khi AI generate code sai → không sửa code trực tiếp → tìm điểm trong Spec chưa rõ ràng → sửa Spec → re-generate.

Code là artifact tạm thời. Spec mới là nguồn sự thật lâu dài.

Nội Dung Chương 6

1

Quy Trình 5 Pha của SDD

Context Discovery → Spec → Plan → Tasks → Implementation → Validation

2

GitHub Spec Kit

Workflow thực tế với 5 Slash Commands — 5 Touchpoints trong GitHub Issues & PRs

3

Cline (Roo Code)

Open Agentic IDE — đọc file, chạy terminal, multi-model, MCP integration

4

DIY SDD

Xây dựng workflow riêng với Markdown thuần — tự do tuyệt đối, không toolkit

5

Hands-on Lab A → Z

Lab A: CRUD đơn giản (45 phút) · Lab B: Import Business Logic phức tạp (60 phút)

6

Evaluation & Reflection

Rubric chấm điểm, bài học từ hai Labs, tổng kết chương



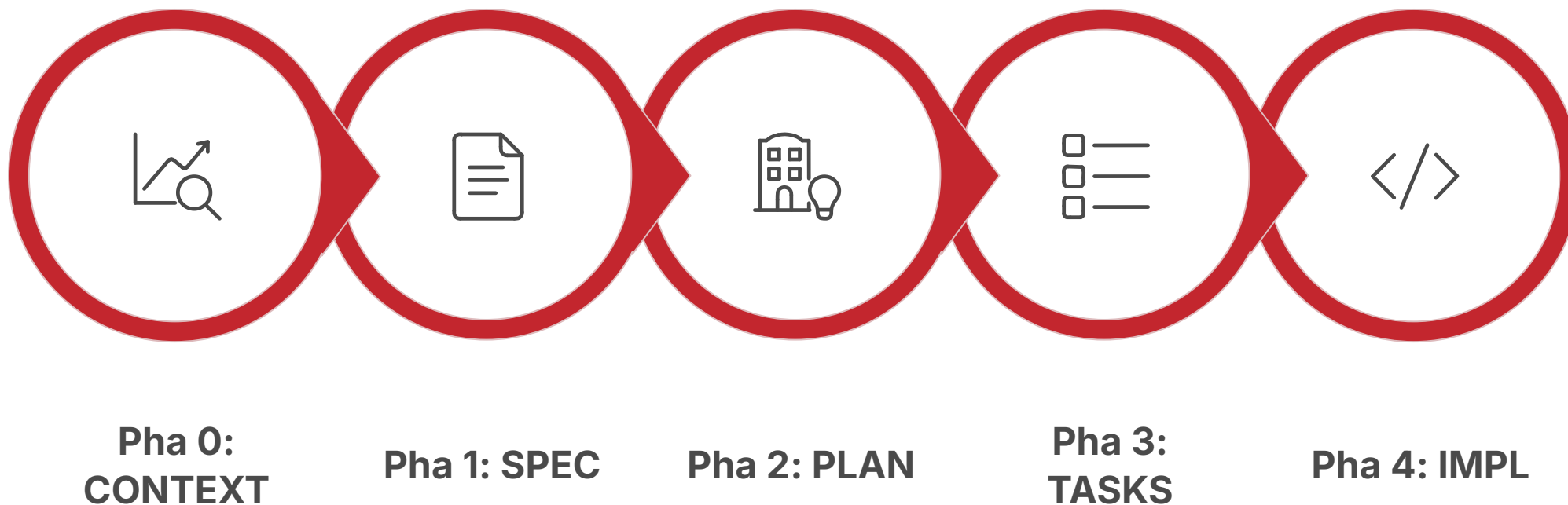
6.1

Quy Trình 5 Pha của SDD

Pipeline tuyến tính với các điểm kiểm tra rõ ràng. Mỗi pha có đầu vào, đầu ra, người chịu trách nhiệm, và Definition of Done cụ thể. Không được phép chuyển sang pha tiếp theo khi pha hiện tại chưa hoàn thành.

5 Pha — Tổng Quan SDD Pipeline

Mỗi pha có đầu vào, đầu ra, và người chịu trách nhiệm rõ ràng — tạo nên một vòng lặp kiểm soát chất lượng từ ý tưởng đến code.



- ❏ Khi validation thất bại → không sửa code trực tiếp. Quay về Spec, tìm điểm chưa rõ, sửa nguồn — rồi re-generate. Đây là nguyên tắc cốt lõi: **Fix the Spec, not the Code.**

Bảng Tổng Quan 5 Pha

Pha	Tên	Actor	Input	Output	Definition of Done
0	Context Discovery	Human	User stories, domain knowledge	Context document	Team hiểu domain & constraints
1	Specification	Human	Context doc, user stories	SPEC.md (APPROVED)	AI review passed, spec locked
2	Planning	AI	SPEC.md	PLAN.md	Human approved arch + risks
3	Task Decomp.	AI	PLAN.md	TASKS.md	Tasks atomic & independent
4	Implementation	AI	TASKS.md + SPEC.md	Source code + tests	All tests green
5	Validation	Both	Code + SPEC.md	Validation report	Code ↔ Spec 100% match

 **Rule quan trọng nhất** Không được phép chuyển sang pha tiếp theo khi pha hiện tại chưa hoàn thành — không có "gần xong", không có "sẽ update sau".

Pha 0 — Context Discovery

Đây là pha thường bị bỏ qua nhất vì nó không tạo ra file code nào. Nhưng đây chính là nguyên nhân số một khiến dự án AI-assisted thất bại: team đã viết spec và nhờ AI implement mà không thực sự hiểu domain. AI implement đúng spec nhưng spec lại sai về mặt business.

Thời lượng thực tế

- Feature nhỏ: 30 phút là đủ
- Module lớn: cần 2–3 buổi workshop
- Điểm quan trọng: ra khỏi pha này với Context Document — không phải trong đầu, phải là file viết ra

Dấu hiệu chưa xong

- Vẫn còn Open Questions chưa được trả lời
- Team chưa đồng thuận về domain terms
- Constraints chưa được liệt kê đầy đủ
- Không ai biết ai là Decision Maker cuối cùng

Template CONTEXT.md

1 Problem Statement

Vấn đề thực sự là gì? User đang bị đau ở đâu? Tránh solution thinking ở bước này. Chỉ mô tả pain.

2 Domain Knowledge

Các thuật ngữ domain-specific mà AI cần biết. Các quy tắc nghiệp vụ bất thành văn.

3 Stakeholders

Ai được lợi? Ai chịu ảnh hưởng? Ai có quyền quyết định?

4 Constraints (không thể thay đổi)

Tech, Business, Time — ràng buộc cứng.

5 Assumptions & Open Questions

Mỗi assumption là một rủi ro nếu sai. Open Questions cần clarify với stakeholder trước khi viết spec.

Pha 1 — Specification

Đây là pha con người làm chủ hoàn toàn. Dựa trên Context Document từ Pha 0, bạn viết SPEC.md theo đúng cấu trúc 8 thành phần và EARS Notation đã học ở Chương 5. Sau khi viết xong draft, mới đưa AI vào để review — không sớm hơn.

Checklist Pha 1 — Definition of Done

- SPEC.md có đủ 8 thành phần (7 core + Out of Scope)
- Mỗi requirement dùng EARS Notation (WHEN/WHILE/WHERE/SHALL)
- AI đã review và tìm logic gaps — mọi gaps đã được resolve
- Không còn Open Questions từ Pha 0
- Spec đã được team lead hoặc product owner approve
- SPEC.md version 1.0.0 đã commit vào git với tag
- "Out of Scope" section rõ ràng, không ambiguous

📌 **Điểm mấu chốt** Pha 1 kết thúc khi spec được lock với trạng thái APPROVED. Không có "gần xong", không có "sẽ update sau". Spec phải cứng trước khi AI bắt đầu làm việc.

Pha 2 — Planning (AI tạo PLAN.md)

Pha 2 là lần đầu tiên AI tham gia chủ động với vai trò kiến trúc sư. AI đọc SPEC.md và tạo ra PLAN.md — một tài liệu mô tả cách tiếp cận kỹ thuật, các component cần tạo, dependency giữa chúng, và risk assessment. Con người review và approve trước khi đi tiếp.

PLAN.md phải có

- Architectural Approach — pattern, design pattern, lý do
- Components — tên, trách nhiệm, interface
- Data Flow — user input → processing → storage → response
- Dependencies — thứ tự implement, external deps
- Risks & Mitigations — ít nhất 3 rủi ro kỹ thuật
- Questions for Human — những điểm spec còn mơ hồ

Tại sao cần Pha 2 riêng biệt?

"Design trước, code sau" luôn cho kết quả tốt hơn. AI có xu hướng bắt đầu code ngay khi được đưa spec — giống như nhà thầu xây móng trước khi có bản vẽ.

PLAN.md buộc AI phải "nghĩ" trước khi "làm".

PLAN.md — Bảng Components & Data Flow

Component	Trách nhiệm	File
ReviewRouter	HTTP endpoints, validation	src/reviews/router.py
ReviewService	Business logic từ SPEC §3	src/reviews/service.py
ReviewRepository	DB CRUD + aggregate query	src/reviews/repository.py
BlacklistService	Filter inappropriate content	src/shared/blacklist.py
AggregateUpdateJob	Background rating recalculation	src/jobs/aggregate_update.py

Data Flow

```
POST /reviews → ReviewRouter (validate input)
→ ReviewService.create_review()
→ OrderRepository.verify_purchase() [check buyer đã mua]
→ ReviewRepository.find_by_order() [check duplicate]
→ BlacklistService.scan() [check content]
→ ReviewRepository.create() [persist]
→ Response 201 Created
(async) → AggregateUpdateJob.enqueue()
```

PLAN.md — Bảng Risks & Mitigations

Rủi ro	Khả năng	Impact	Mitigation
Race condition: double submit	High	High	DB unique constraint
Aggregate lag > 5 min	Med	Med	Redis cache + job queue
Blacklist false positives	Med	Low	Whitelist + user appeal

📌 **"Questions for Human" là vàng.** Đây là nơi AI cho bạn biết những chỗ spec còn mơ hồ. Nếu AI không có câu hỏi nào, hoặc spec của bạn rất tốt, hoặc AI đang assume — hãy hỏi lại: *"Bạn đã assume điều gì?"* Mỗi assumption ẩn là một bug tiềm ẩn cho nó.

Questions for Human — ví dụ

- AggregateUpdateJob: dùng Celery hay Kafka? (spec không chỉ định)
- BlacklistService: hardcoded list hay dynamic từ DB?
- Spec §3 nói "cập nhật trong 5 phút" — eventual consistency hay realtime với cache invalidation?



"Mỗi assumption ẩn là một bug tiềm ẩn cho nó."

Nếu AI không đặt câu hỏi sau khi đọc PLAN.md — hãy chủ động hỏi nó: "Bạn đã assume điều gì trong quá trình tạo plan này?"

PLAYBOOK SDD + ADD — CHƯƠNG 6

Pha 3 — Task Decomposition

Từ PLAN.md, AI tạo TASKS.md — danh sách các task cụ thể, có thể implement độc lập, với acceptance criteria rõ ràng cho từng task. Đây là bước chuyển từ "what to build" sang "how to build it step by step".

Atomic

Không chia nhỏ hơn được

Independent

Implement được mà không phụ thuộc task chứa xong

Verifiable

Có test case cụ thể để biết đã xong

 **Prompt Pha 3 — Tạo TASKS.md** Doc PLAN.md và SPEC.md, tạo TASKS.md — danh sách tasks có thể implement độc lập. Format ID: T001, T002... · Estimated time · Dependencies · EARS spec refs · Done criteria. **Tối đa 4 giờ/task. Nếu task > 4h, chia nhỏ hơn.**

TASKS.md — Product Review Feature

ID	Task	Est	Deps	Spec Refs	Done When
T001	Tạo DB migration reviews table	1h	—	§5 Data	Migration runs, rollback OK
T002	Implement BlacklistService	2h	—	§3 Unwanted	10 test cases pass
T003	Implement ReviewRepository	3h	T001	§5 Data	CRUD ops + aggregate query OK
T004	Implement ReviewService	4h	T002,T003	§3 Func.	All business rules enforced
T005	Implement ReviewRouter	2h	T004	§3 Func.	All endpoints return correct HTTP
T006	Implement AggregateUpdateJob	3h	T003	§4 Non-func	Rating updates within 5 min
T007	Integration tests	2h	T005,T006	§7 Accept.	All AC checklist items pass
T008	API documentation	1h	T005	—	OpenAPI spec generated

Pha 4 — Implementation

Đây là pha AI làm việc nhiều nhất — nhưng không phải theo nghĩa "chat và nhận code". AI implement từng task trong TASKS.md, theo đúng thứ tự dependency, với SPEC.md và PLAN.md luôn trong context. Không có task nào được implement mà không có spec refs rõ ràng.



Prompt mẫu — Implement Task T004: ReviewService

- Implement ReviewService theo ĐÚNG spec §3
- Dùng dependency injection — không instantiate deps bên trong
- Thêm EARS tag comment cho mỗi business rule
- Xử lý TẤT CẢ Unwanted patterns trong spec §6
- Type hints bắt buộc cho tất cả method signatures
- Tạo đồng thời: tests/reviews/test_service.py
- KHÔNG implement những gì nằm trong Out of Scope (SPEC §8)

6.1.5 — NGUYÊN TẮC CỐT LỖI

"Fix the Spec, not the Code."

✗ WRONG — Sửa code trực tiếp

AI generate: if rating not in range(1, 5):

Dev sửa tay: if rating not in range(1, 6):

→ Code diverge khỏi spec, không ai biết tại sao

✓ RIGHT — Trace về Spec

Test fails → Check SPEC §3: "rating 1–5 (integer)"

Spec rõ nhưng AI dùng range() sai

Update prompt → Re-generate → Correct

Rule: KHÔNG có dòng code nào không truy được về spec.

Pha 5 — Validation (Kiểm tra Code vs. Spec)

Pha cuối cùng là formal verification: đọc code và spec song song, xác nhận mọi requirement đều được implement đúng. Đây không phải code review thông thường — đây là compliance check.

Spec Section	Requirement (tóm tắt)	Code File/Line	Test Name	Status
§3 WHEN submit	Validate rating 1–5	service.py:L45	test_rating_boundary_valid	Pass
§3 WHEN submit	Validate comment 10–500	service.py:L52	test_comment_length_boundaries	Pass
§3 Unwanted buyer	Reject non-buyer review	service.py:L61	test_non_buyer_rejected	Pass
§3 Unwanted dup	Reject duplicate review	service.py:L72	test_duplicate_review_rejected	Pass
§8 Out of Scope	NO moderation queue	(verify absent)	test_no_moderation_endpoint	Pass

 **Out of Scope cũng cần Validation** Nhiều team chỉ verify "những gì đã làm". SDD cũng verify "những gì không được làm". grep codebase tìm code không có trong spec → nếu AI "helpful" implement thêm feature ngoài scope → remove ngay.

Bài Tập — Mapping SDD Phases

Bài tập 6.1.A — Phase Identification ★★

Cho đoạn hội thoại sau — xác định mỗi message thuộc pha nào và tại sao developer này đang làm sai:

- Dev: *"Viết cho tôi authentication module cho app Node.js."*
- AI: [Generate code 200 dòng không có spec]
- Dev: *"Code bị lỗi, sửa đi."* [Sửa code trực tiếp]
- Dev: *"Thêm 2FA vào đi."* [Feature mới không có spec]

→ Pha nào bị skip? Sai lầm cụ thể là gì? SDD đúng trông như thế nào?

Bài tập 6.1.B — PLAN.md Review ★★★

Viết SPEC.md ngắn (Detailed level) cho tính năng "Password Reset". Sau đó đưa cho Cline với prompt Pha 2 để generate PLAN.md.

Review output của AI: risks nào nó tìm được? Questions nào nó đặt ra? Bạn đồng ý với architectural approach không?



6.2

GitHub Spec Kit — Workflow thực tế với Slash Commands

Workflow thực tế với Slash Commands — đưa SDD pipeline trực tiếp vào GitHub Issues và Pull Requests. 5 Slash Commands, 5 Touchpoints, mỗi lần chuyển pha đều có human-in-the-loop.

GitHub Spec Kit — Cài Đặt Và Cấu Hình

Trước khi dùng slash commands, team cần chuẩn hóa môi trường repository. Mục tiêu của bước này là đảm bảo mọi thành viên chạy chung một workflow: cùng templates, cùng actions, cùng cấu hình model và guardrails.

Checklist cài đặt

- Thêm thư mục `.github/ISSUE_TEMPLATE/` cho feature spec template
- Thêm workflows `.github/workflows/speckit-*.yml`
- Thêm prompts trong `.github/speckit/prompts/` theo từng pha
- Thiết lập quyền cho GitHub Actions: tạo comment, tạo PR, đọc issue

Cấu hình bắt buộc

- `.github/speckit/constitution.md` — AGENTS.md của project
- `.github/speckit/config.yml` — model, token limits, retries
- Secrets cho CI (API keys) lưu ở GitHub Secrets, không hardcode
- Quy định nhãn issue: `spec:draft`, `spec:approved`, `plan:approved`

📌 **Nguyên tắc setup** Cài đặt một lần, dùng lâu dài. Khi cấu hình chưa chuẩn, slash commands sẽ tạo output không đồng nhất giữa các project.

GitHub Spec Kit — Triết Lý Thiết Kế

GitHub Spec Kit là tập hợp workflows, templates, và GitHub Actions được thiết kế để đưa SDD pipeline trực tiếp vào GitHub Issues và Pull Requests. Thay vì quản lý SPEC.md, PLAN.md, TASKS.md thủ công, Spec Kit cung cấp các slash commands — những điểm chạm (touchpoints) chuẩn hóa giữa người và AI.

Mỗi slash command là một "handoff point" — nơi con người chuyển giao quyền kiểm soát cho AI một cách có chủ đích, với context được chuẩn bị sẵn. Không có gì xảy ra mà không có ý chí của người dùng.

5 Slash Commands — 5 Touchpoints

Command	Pha	Actor kích hoạt	AI làm gì
/speckit.constitution	Setup	Tech lead, once	Tạo AGENTS.md từ project description + tech stack
/speckit.specify	Pha 1	Developer / PM	Review spec draft, tìm gaps, tạo SPEC.md version cuối
/speckit.plan	Pha 2	Developer	Đọc SPEC.md, tạo PLAN.md với arch + risks + questions
/speckit.tasks	Pha 3	Developer (sau approve)	Đọc PLAN.md, tạo TASKS.md với atomic tasks và deps
/speckit.implement	Pha 4	Developer (per task)	Implement task cụ thể + tests, tạo PR

 **Slash Commands là Touchpoints, không phải automation** Không command nào là "bấm và quên". Sau **/speckit.specify** → người dùng đọc và APPROVE spec trước khi tiếp tục. Sau **/speckit.plan** → người dùng review PLAN.md, trả lời Questions for Human. AI chờ. Không có gì tự động sang pha tiếp theo — đây là cách SDD đảm bảo human-in-the-loop.

/speckit.specify & /speckit.plan

/speckit.specify — Từ ý tưởng đến SPEC.md

Command quan trọng nhất. Gọi khi đã có draft spec (hoặc chỉ có user story). AI sẽ:

- Expand thành SPEC.md đầy đủ 8 thành phần
- Thêm EARS notation cho mỗi requirement
- Tìm edge cases: xóa địa chỉ mặc định → làm sao?
- Thêm Out of Scope rõ ràng
- Post spec vào Issue dưới dạng comment + attach file

/speckit.plan — AI kiến trúc hóa

Trigger sau khi SPEC.md đã được approve. AI sẽ:

- Fetch SPEC.md từ Issue
- Đọc constitution.md (tech stack, patterns)
- Tạo PLAN.md: arch, components, data flow, deps, risks
- Post PLAN.md vào Issue dưới dạng comment
- Tag reviewer để approve

Sau khi plan được approve → dev có thể chạy /speckit.tasks

/speckit.tasks & /speckit.implement

/speckit.tasks — Atomic task breakdown

Trigger sau khi PLAN.md approved. AI sẽ:

- Đọc SPEC.md + PLAN.md
- Tạo TASKS.md với atomic tasks (max 4h/task)
- Tạo GitHub sub-issues cho mỗi task tự động
- Gán dependencies bằng GitHub issue links

Issue #42 (feature): "Shipping Address"

#43 (T001): "Create DB migration"

#44 (T002): "Implement Repository"

#45 (T003): "Implement Service [deps: #44]"

/speckit.implement — Code theo spec

Trigger trong task sub-issue. AI sẽ:

- Đọc task definition, fetch SPEC.md và PLAN.md
- Đọc code đã có từ dependency tasks
- Generate code + tests với EARS tags
- Tạo Pull Request tự động với traceability report

PR description: Task ID · Spec refs · Test coverage · Out of Scope verified

Full Walkthrough — Từ Init Đến Complete

Ngày 1 — Setup & Specify

- PM tạo GitHub Issue: "Feature: Shipping Address"
- Dev đọc, hiểu domain → viết Context trong Issue body
- Dev trigger: `/speckit.constitution` (chỉ lần đầu)
- Dev viết draft spec → trigger: `/speckit.specify`
- Review output, resolve 3 gaps AI tìm được
- Approve spec → Issue labeled "spec:approved"

Ngày 2-4 — Plan, Tasks, Implement

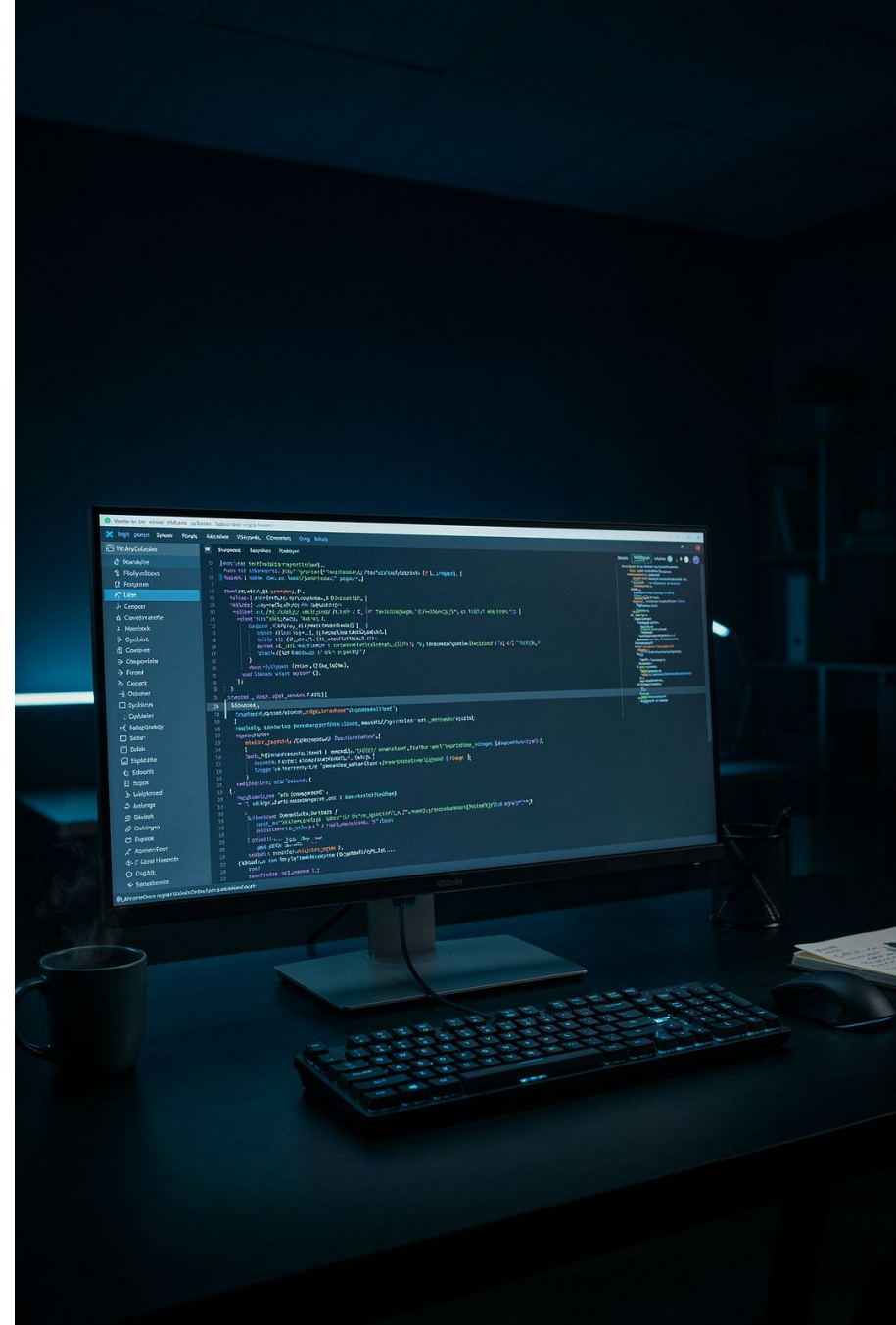
- `/speckit.plan #42` → AI tạo PLAN.md
- Review plan, trả lời 2 Questions for Human
- Approve plan → `/speckit.tasks #42` → 5 sub-issues
- Mỗi task sub-issue: `/speckit.implement` → code + PR
- Review PR: check traceability, approve nếu OK → merge
- Tất cả 5 tasks merged → Run validation → Close #42

✅ **Timeline & Audit Trail** 4 ngày cho feature với full spec trail. Mọi quyết định — ai làm gì, khi nào, dựa trên spec nào — đều được lưu trong GitHub Issues. Traceability hoàn hảo.

6.3

Cline (Roo Code) — Sức mạnh của Open Agentic IDE

Open Agentic IDE — đối cực của Spec Kit. Chạy ngay trong VSCode, đọc file, chạy terminal, gọi MCP servers, thực thi kế hoạch multi-step mà không cần copy-paste từng đoạn code.



Cline vs. Các Công Cụ Khác

Tiêu chí	Cline (Roo Code)	GitHub Copilot Chat	Cursor AI	Kiro IDE
Open source	MIT License	Proprietary	Proprietary	AWS-controlled
Agentic (multi-step)	Native	Limited	Composer	Native
Multi-model support	Anthropic/OpenAI/Ollama	Copilot only	Multiple	AWS Bedrock
MCP integration	Full	None	Partial	Limited
File system access	Full read/write	Read only	Full	Full
Terminal execution	With approval	No	Yes	Yes
Cost model	Pay-per-token (API)	Subscription	Subscription	AWS pricing
Vendor lock-in	Lowest	GitHub/MS	Medium	AWS

Cline Tự Động Đọc File & Chạy Terminal

Điểm khác biệt lớn nhất của Cline so với AI chatbot thông thường là khả năng thực thi — không chỉ suggest. Ví dụ: giao Cline implement Task T004 với full autonomy.

Cline tự động thực hiện từng bước

01	02
<code>read_file("TASKS.md")</code> → hiểu T004 requirements	<code>read_file("SPEC.md")</code> → load business rules
03	04
<code>read_file("src/reviews/repository.py")</code> → đọc interface	<code>write_file("src/reviews/service.py")</code> → implement
05	06
<code>write_file("tests/reviews/test_service.py")</code> → tests	<code>execute_command("pytest tests/reviews/ -v")</code> → run
07	08
[nếu fail] đọc output, fix service.py, re-run cho đến khi pass	Report: "All 15 tests passed. Coverage: 94%."

 **Developer chỉ cần approve các "dangerous" actions** `write_file`, `execute_command` khi Cline xin phép — toàn bộ còn lại Cline tự xử lý.

Multi-Model — Chọn Model Đúng Cho Đúng Task

Task trong SDD	Model khuyến nghị	Lý do	Approx cost
Pha 0: Context Discovery	Claude Sonnet	Cần reasoning, domain understanding	\$0.003/1k tokens
Pha 1: Spec review + gaps	Claude Sonnet	Logic analysis, contradiction detect	\$0.003/1k tokens
Pha 2: Planning (PLAN.md)	Claude Sonnet	Architecture decisions cần depth	\$0.003/1k tokens
Pha 3: Task decomp.	Claude Haiku	Structured breakdown, không cần depth	\$0.00025/1k tokens
Pha 4: Boilerplate code	Claude Haiku / GPT4o-mini	Simple patterns, CRUD generation	\$0.00015/1k tokens
Pha 4: Business logic	Claude Sonnet	Complex logic cần reasoning	\$0.003/1k tokens
Pha 5: Validation	Claude Sonnet	Careful comparison cần accuracy	\$0.003/1k tokens
Offline / confidential	Llama 3 via Ollama	Data không ra ngoài machine	Free (local GPU)

Cline + MCP = Agentic SDD Hoàn Chỉnh

Kết nối Cline với MCP servers tạo ra workflow agentic thực sự: AI không chỉ generate code — nó đọc Jira để biết requirement, đọc GitHub để biết existing code, viết code, chạy tests, và tạo PR. Tất cả mà không cần bạn switch tab.

MCP Servers cho SDD Workflow

- @modelcontextprotocol/server-github — đọc/tạo Issues, PRs
- jira-mcp — đọc tickets, update status
- @modelcontextprotocol/server-filesystem — đọc/ghi files

Kịch bản agentic đầy đủ

Bạn nói với Cline:

"Đọc Jira ticket PROJ-123, implement theo spec, tạo PR và link vào ticket."

Cline sẽ tự động orchestrate toàn bộ: fetch ticket → read spec → implement → test → create PR → update Jira.

Không có MCP → Cline chỉ là AI code editor thông minh. Với MCP → Cline thực hiện multi-step workflow hoàn chỉnh. Đây chính là tầm nhìn của SDD: **con người design ý định, AI thực thi toàn bộ.**

Bài Tập — Cline Agentic Workflow

Bài tập 6.3.A — Single-task agent run

- Chọn 1 task từ TASKS.md (không quá 2h)
- Dùng Cline đọc SPEC.md + PLAN.md + task definition
- Yêu cầu Cline implement + viết test + chạy test tự động
- Ghi lại trace: command nào chạy, lỗi nào gặp, cách self-fix

Bài tập 6.3.B — MCP-enhanced workflow

- Kết nối Cline với MCP GitHub hoặc filesystem server
- Chạy kịch bản: đọc issue → implement task → tạo PR description
- Đánh giá output theo 3 tiêu chí: traceability, test coverage, out-of-scope compliance
- Viết reflection ngắn: điểm mạnh/yếu khi để agent tự chạy multi-step

📌 **Mục tiêu** Không phải "ra code nhanh nhất", mà là kiểm soát được hành vi agent qua spec artifacts và approval points.

6.4

DIY SDD

Xây dựng workflow riêng không cần toolkit — chỉ với Markdown files, git, và bất kỳ AI chat nào. Bản chất SDD không phụ thuộc vào công cụ, mà phụ thuộc vào nguyên lý.



Thư Mục .sdd/ — Tiêu Chuẩn SDD

Quy ước thư mục .sdd/ đặt tất cả spec artifacts trong một nơi, tách biệt với source code nhưng nằm cùng repo. Dấu chấm đầu tên tránh nhầm lẫn với source code, đồng thời truyền tín hiệu rõ ràng: đây là metadata của dự án.

Cấu trúc thư mục

```
.sdd/
├── README.md      # Hướng dẫn SDD trong project
├── constitution.md # = AGENTS.md: rules cho AI
├── specs/
│   ├── _template.md # Template tạo spec mới
│   ├── feature-auth/
│   │   ├── SPEC.md # Spec chính (locked khi approved)
│   │   ├── CONTEXT.md # Context discovery output
│   │   ├── PLAN.md # AI-generated plan
│   │   ├── TASKS.md # Task breakdown
│   │   └── CHANGELOG.md
│   └── feature-cart/
├── reviews/      # AI spec review outputs
└── metrics/      # Token usage, spec quality
```

Tại sao .sdd/ vào git?

- Spec là source of truth → phải được version control như code
- git diff SPEC.md → xem spec thay đổi thế nào qua thời gian
- git blame SPEC.md → ai thay đổi dòng nào, khi nào
- git revert → rollback spec khi quyết định thay đổi
- Pull Request cho spec → review process giống code review

Biến Markdown Thành "Bản Giao Kèo" Với AI

Mỗi khi bắt đầu AI session mới, bạn không chat tự nhiên — bạn trình bày contract và yêu cầu AI "ký" (acknowledge) trước khi làm việc. Điều này áp dụng cho mọi AI: ChatGPT, Claude, Gemini, hay bất kỳ model nào.

System Prompt chuẩn — Đầu mỗi AI session

- === CONTRACT === — khai báo bối cảnh
- Paste nội dung constitution.md (rules)
- CURRENT TASK: Chúng ta đang ở Pha N. Nhiệm vụ cụ thể.
- CONTEXT FILES: Paste SPEC.md, PLAN.md nếu có
- CONSTRAINTS: KHÔNG implement Out of Scope · KHÔNG assume
- Mọi code phải có EARS tag tham chiếu về SPEC section

📌 **Bước "Xác nhận" cuối prompt — rất quan trọng** Yêu cầu AI: *"Xác nhận bạn đã đọc contract. Tóm tắt nhiệm vụ của bạn trong 3 bullet points trước khi bắt đầu."* Nếu AI tóm tắt sai → bạn biết nó đã hiểu sai context và có thể điều chỉnh ngay.

DIY Workflow Hoàn Chỉnh — Password Reset

Bước 1–4: Setup & Spec

- Tạo structure: `mkdir -p .sdd/specs/feature-password-reset`
- Viết `CONTEXT.md` (30 phút, tự làm) — domain knowledge, constraints
- Viết `SPEC.md` draft (45 phút) — 8-component template, EARS notation
- AI Review session → paste contract + yêu cầu tìm logic gaps
- Update `SPEC.md`, lock: `git commit + git tag spec/password-reset/v1.0.0`

Bước 5–9: Plan, Tasks, Implement

- AI tạo `PLAN.md` (Claude.ai session mới) → paste contract với Pha 2
- Copy output vào file, review và approve
- AI tạo `TASKS.md` (session mới) → paste `SPEC.md` + `PLAN.md`
- Implement trong VSCode với Cursor/Cline — dùng `SPEC.md` + `PLAN.md` trong context (@file)
- Validation: AI đọc code + spec, tạo traceability matrix → merge nếu pass

Naming Conventions & Git Strategy

Artifact	Naming	Location	Ví dụ
Feature spec	feature-{name}/SPEC.md	.sdd/specs/	feature-cart/SPEC.md
Bugfix spec	fix-{issue-id}/SPEC.md	.sdd/specs/	fix-gh-234/SPEC.md
Plan	PLAN.md (trong folder)	.sdd/specs/{name}/	feature-cart/PLAN.md
Tasks	TASKS.md (trong folder)	.sdd/specs/{name}/	feature-cart/TASKS.md
AI review	{name}-review-{date}.md	.sdd/reviews/	cart-review-2025-01-20.md
Git tag	spec/{name}/v{semver}	git tag	spec/cart/v1.0.0
Git commit	spec({name}): {action}	commit msg	spec(cart): approve v1.0.0
Branch	spec/{name}-draft	git branch	spec/cart-draft

6.5

Hands-on Lab A → Z

Bài thực hành trung tâm của chương — chạy toàn bộ SDD pipeline cho hai tính năng với độ phức tạp khác nhau. Lab B là "bài test thực sự".



Lab A — CRUD Đơn Giản (45 phút)

Feature: Quản lý Categories cho blog. Simple CRUD, không có business logic phức tạp. Mục đích: làm quen với SDD pipeline trong môi trường "an toàn" trước khi tackle Lab B.

User Story ban đầu (rough)

- Tạo, đọc, sửa, xóa categories
- Category có: name (unique), slug (auto-gen), description
- Không thể xóa category đang có bài viết
- Danh sách sorted by name

Pipeline Steps (30 phút với Cline)

- **Pha 2** — "Đọc SPEC.md và tạo PLAN.md. Chú ý: slug auto-generation logic cần rõ ràng."
- **Pha 3** — "Đọc PLAN.md, tạo TASKS.md. Max 4h/task. T001 (migration) là task đầu tiên."
- **Pha 4** — "Implement {task_id}. Đọc SPEC.md và PLAN.md. EARS tags bắt buộc. Generate tests cùng lúc."
- **Pha 5** — "Tạo traceability matrix: mỗi EARS requirement → code line → test name → pass/fail."

Lab A — SPEC.md Category Management

§3 Functional Requirements (EARS)

- WHEN admin POST /categories với valid data, THE system SHALL tạo category và return 201 + object
- WHEN admin GET /categories, THE system SHALL return danh sách sorted by name ascending
- WHEN admin PUT /categories/{id}, THE system SHALL cập nhật và return 200 + updated object
- WHEN admin DELETE /categories/{id}, THE system SHALL xóa và return 204

§6 Error Handling (Unwanted)

- WHERE POST với name đã tồn tại → 409 Conflict
- WHERE DELETE category có bài viết → 422 + "Cannot delete: category has {n} posts. Reassign first."
- WHERE PUT/DELETE với id không tồn tại → 404

§8 Out of Scope

- Không có nested/hierarchical categories
- Không có soft delete (hard delete only)
- Không có search/filter trong sprint này

Lab B — Import CSV Với Business Logic Phức Tạp (60 phút)

Đây là bài test thực sự. Tính năng Import Data có đủ độ phức tạp để bẫy tất cả những gì có thể sai nếu Planning và Task Decomposition bị bỏ qua: format validation, error logging, partial success, rollback strategy. Những vấn đề này không hiện ra khi nhìn vào user story — chỉ hiện ra khi bạn spec kỹ.

User Story ban đầu (từ PM) — tất cả những gì PM cung cấp

Admin có thể upload file CSV để import hàng loạt sản phẩm. Requirements: upload CSV, parse và import vào database, báo cáo số lượng thành công/thất bại, xử lý lỗi định dạng.

← Đây là tất cả. Phần tiếp theo: bạn phải tự phát hiện tất cả những gì còn thiếu qua SDD process.

Lab B — Câu Hỏi Context Discovery

Danh sách câu hỏi bạn phải trả lời trước khi viết spec. Chúng không xuất hiện trong user story — bạn phải tự phát hiện:

Câu hỏi	Nếu không hỏi → Bug
"CSV format cụ thể thế nào? Header row có không?"	AI đoán format → import sai toàn bộ
"Nếu 1 row lỗi trong 1000 rows — rollback hay skip?"	Hành vi undefined → inconsistent data
"Duplicate product (same SKU) → update hay error?"	Silent data override hoặc false rejection
"File size limit?"	Memory exhaustion khi upload 500MB CSV
"Log ở đâu? Admin xem được không? Trong bao lâu?"	Log mất sau restart, không debug được
"Import async hay sync? Timeout?"	30s timeout khi import 50,000 rows
"Validation per-row hay per-field?"	Row 1 lỗi field 2 → message không rõ

Lab B — SPEC.md Product CSV Import

§3 Functional — CSV Format & Upload Flow

- Accept CSV: UTF-8, comma delimiter, header row bắt buộc: sku, name, price, stock, category_slug
- Tối đa 10,000 rows mỗi file · File size tối đa: 10MB
- WHEN admin POST /imports/csv với valid file, THE system SHALL: lưu file vào temp, tạo import_job (status=pending), return 202 Accepted + job_id, process async
- Processing Strategy: partial success — mỗi row xử lý độc lập, không transaction toàn bộ file
- WHERE SKU đã tồn tại → upsert (không lỗi, không duplicate)

§6 Error Handling — Unwanted Patterns

- WHERE file không phải CSV hoặc > 10MB → 400 ngay (không process)
- WHERE header row thiếu cột bắt buộc → fail toàn bộ job + error message
- WHERE một row thiếu trường → skip row, ghi error: "Row {n}: Missing {field}"
- WHERE price không phải số dương → skip + "Row {n}: Invalid price"
- WHERE error_count = total_rows → status = "failed" + email alert admin
- WHERE 3 concurrent jobs đang running → 429 Too Many Requests

Lab A vs Lab B — Độ Phức Tạp Thiết Kế

Vấn đề thiết kế	Lab A (CRUD)	Lab B (Import)
Async processing	Không cần	Bắt buộc — background job queue
Transaction strategy	Per-request transaction	Per-row, không global rollback
Error storage	Không cần	import_errors table + downloadable
Concurrency control	Không cần	Semaphore/lock cho 3-job limit
File storage	Không cần	Temp storage + cleanup job
Progress tracking	Không cần	Real-time counters + polling
Testing complexity	Unit tests đủ	Integration + end-to-end bắt buộc

⚠ **Bài học từ Lab B** "Import CSV" nghe đơn giản nhưng chứa đựng 6 quyết định thiết kế quan trọng. Không có Context Discovery → miss file size limit, miss concurrent control. Không có PLAN.md → AI implement sync thay vì async → timeout ở production. Đây chính xác là lý do SDD tồn tại: complexity luôn cao hơn vẻ ngoài.

Lab B — TASKS.md Product CSV Import (12 tasks, ~28h)

ID	Task	Est	Dependencies
T001	DB migration: import_jobs table	1h	—
T002	DB migration: import_errors table	1h	T001
T003	FileValidationService (size, ext)	2h	—
T004	CSVParserService (header + rows)	3h	T003
T005	RowValidatorService (per-field)	3h	—
T006	ImportJobRepository	2h	T001,T002
T007	ImportWorker (async background) — critical path	4h	T004,T005,T006
T008	ConcurrencyGuard (3-job limit)	2h	T006
T009	ImportRouter (upload + status)	2h	T007,T008
T010	ErrorExportEndpoint (CSV download)	2h	T006
T011	Integration tests	4h	T009,T010
T012	Admin email alert (all-fail case)	2h	T007

Evaluation Rubric — Chấm Điểm Cả Hai Labs

Tiêu chí	Lab A (CRUD)	Lab B (Import)	Điểm
SPEC đủ 8 thành phần	8/8 sections	8/8 sections	20đ
EARS notation	≥5 SHALL statements	≥12 SHALL statements	20đ
Out of Scope rõ ràng	≥3 items	≥5 items	10đ
PLAN.md có risks	≥2 risks	≥5 risks	10đ
Tasks atomic (≤4h)	All tasks ≤4h	All tasks ≤4h	15đ
Code có EARS tags	≥80% rules tagged	≥80% rules tagged	10đ
Tests cover AC	100% AC covered	100% AC covered	15đ

100

Tổng điểm

40%

Lý thuyết

60%

Thực hành

8

SPEC components

Reflection — Bài Học Từ Hai Labs

Câu hỏi phân tích

- Lab A vs Lab B: Bao nhiêu thời gian bạn dành cho Context Discovery và Spec Writing? Tỷ lệ lý thuyết theo SDD là 30–40% thời gian dự án. Thực tế của bạn là bao nhiêu?
- Khi AI generate PLAN.md cho Lab B, nó hỏi bao nhiêu Questions for Human? So sánh với Lab A. Điều đó nói lên gì về mức độ phức tạp tương đối?

Câu hỏi ứng dụng thực tế

- Nếu bạn bỏ qua Context Discovery và đưa user story trực tiếp vào AI để code — điều gì sẽ thiếu trong sản phẩm cuối? Liệt kê cụ thể.
- Trong môi trường làm việc thực, đồng nghiệp hoặc PM của bạn sẽ phản ứng thế nào khi bạn nói *"Tôi cần 1 ngày để viết spec trước khi code"*? Làm thế nào bạn justify?

📌 Task Decomposition không phải overhead — nó là quality gate. Khi ai đó implement tất cả trong 1 task (không decompose) → context overflow, AI forget spec details → thiếu partial success logic → lỗi production.

Tổng Kết — Ba Con Đường SDD

Cách tiếp cận	Phù hợp với	Điểm mạnh	Điểm yếu
GitHub Spec Kit	Team dùng GitHub, muốn cấu trúc	Traceability tốt, audit trail hoàn hảo	Cần setup, learning curve
Cline (Roo Code)	Developer muốn speed + autonomy	Agentic, multi-model, MCP integration	Cần giám sát, có thể overrun
DIY Markdown	Team nhỏ, linh hoạt, bất kỳ AI	Không vendor lock, đơn giản, tự do	Tự kỷ luật cao hơn cần thiết

Key Takeaways

- 5 pha: Context → Spec → Plan → Tasks → Impl → Validate
- Fix the Spec, not the Code — nguyên tắc bất biến
- Mọi code phải truy được về spec — không exception
- Out of Scope cũng cần validation — không chỉ "những gì đã làm"

Chương tiếp theo — Chương 7

Test-Driven Specification

Given-When-Then (BDD) notation · AI generate test suite trước khi viết code (TDS) · Contract testing cho API · Mutation testing với AI-generated test cases.

"Code là artifact tạm thời. Spec mới là nguồn sự thật lâu dài."

5 pha · Fix the Spec · Human-in-the-loop ở mọi điểm quan trọng

5

SDD Phases

3

Tooling Paths

2

Hands-on Labs

∞

Spec First